



המכללה האקדמית להנדסה ע"ש סמי שמעון

מדור בחינות ומערכת שעות



המחלקה להנדסת תוכנה

7/07/2019
9:00-12:00

מערכות הפעלה

מועד ב'
ד"ר אירינה רבייב

תשע"ט סמסטר ב'

- ענו על ארבע השאלות הבאות על גבי הטופס ובמקומות המתאימים.
- הדפים ישמשו לטיוטה בלבד! דפי הטיוטה ימסרו לגריסה בתום הבחינה.

חומר עזר – מחשבון ללא יכולת תכנות
3 דפי A4 דו-צדדי בכתב יד רגיל, לא מודפס ולא מצולם

יש לענות על כל השאלות!
יש לרשום את התשובות במקום שהוקצה בלבד – כתיב מעבר למקום שהוקצה לא ייבדק!

במידה ואינכם יודעים את התשובה לסעיף כלשהו, רשמו "לא יודע" ותזכו ב-20% מניקוד הסעיף.

בזמן הבחינה יינתן מענה ע"י מרצה/מתרגל לשאלות הבנה בלבד!
במידה וההוראות במבחן ניתנות בלשון זכר – הן מכוונות לנשים ולגברים כאחת.

השאלון מכיל 9 עמודים.

בהצלחה !

=====



שאלה 1 (20 נק')

סעיף א' (10 נק')

נתון קטע הקוד שלהלן:

```
int main(int argc, char* argv[]) {
    int rc;
    int pid;
    int p[2];
    rc = pipe( p );
    printf( "%d \n", getpid() );
    pid = fork();
    if ( pid == 0 ){
        rc = write( p[1], "ABCDEFGHIJKLMNOPQRSTUVWXYZ", 26 );
    }

    char buffer[40];
    rc = read( p[0], buffer, 8 );
    buffer[rc] = '\0';
    printf( "%d %s\n", getpid(), buffer );
    return 0;
}
```

הניחו כי ה-pid של התהליך שמתחיל להריץ את main הוא 8695, וכי pids ניתנים באופן סדרתי, ואין עוד תהליכים במערכת. כמו כן, הניחו שכל קריאות המערכת (system calls) הצליחו.

מהו פלט התוכנית? במידה ויש מספר אפשרויות, ציינו את כולן.

אפשרות 1:

8695
8695 ABCDEFGH
8696 IJKLMNOP

אפשרות 2:

8695
8696 ABCDEFGH
8695 IJKLMNOP

סעיף ב' (10 נק')

נתון קטע קוד שלהלן:

```
int main(int argc, char* argv[]) {  
    int a = 0;  
    int pid = fork();  
    a++;  
    if (pid == 0) {  
        pid = fork();  
        a++;  
    } else {  
        a++;  
    }  
    printf("Hello!\n");  
    printf("a is %d\n", a);  
    return 0;  
}
```

א. בהנחה שכל קריאות `fork()` הצליחו, כמה פעמים תודפס הודעה `"Hello!\n"`?
תשובה: 3

ב. מהו הערך הגדול ביותר של `a` שיודפס על המסך?
תשובה: 2

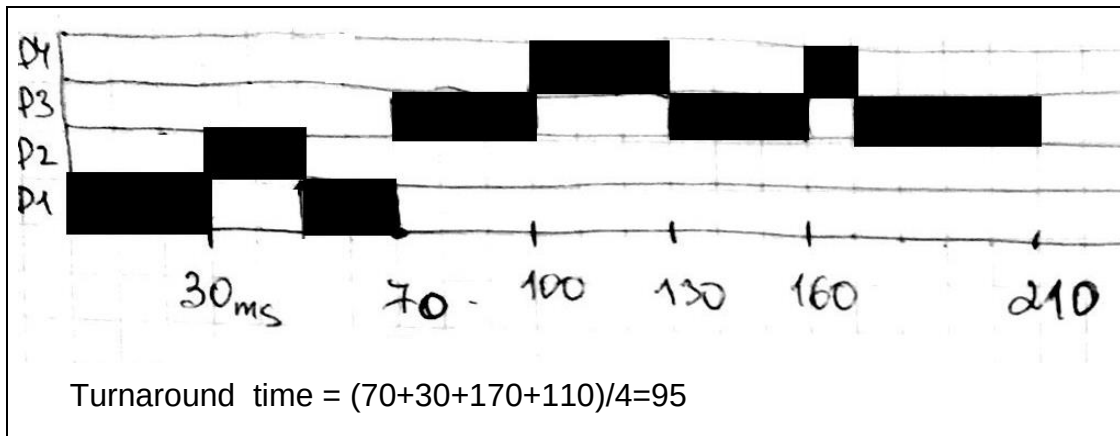
שאלה 2 (20 נק')

ארבע משימות אצווה (batch jobs) P1, P2, P3, P4 מגיעות למערכת.
זמני ההגעה, זמני ה-CPU burst ועדיפויות שלהן נתונים בטבלה (1 היא העדיפות הגבוהה ביותר, 4 היא העדיפות הנמוכה ביותר).

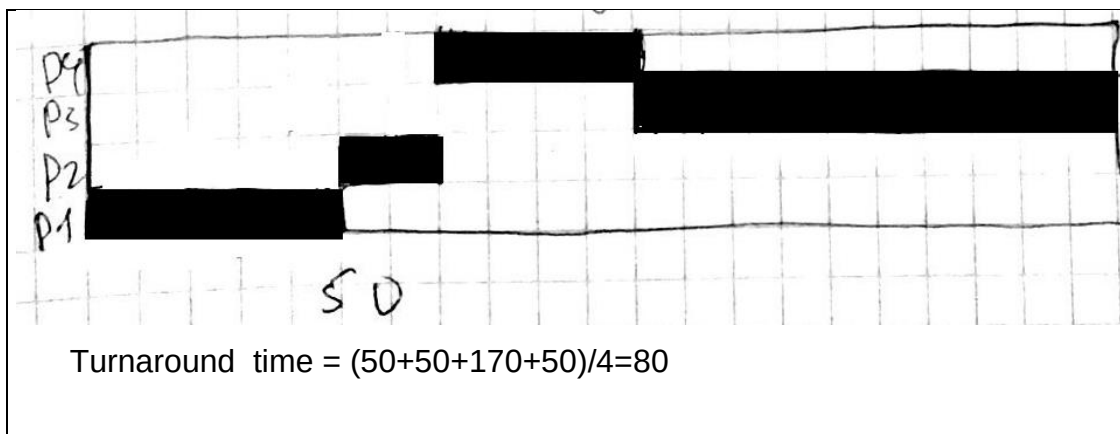
Process	Burst Time	Priority	Arrival Time
P1	50 ms	4	0 ms
P2	20 ms	1	20 ms
P3	100 ms	3	40 ms
P4	40 ms	2	60 ms

עבור כל אחד מאלגוריתמי התזמון הבאים סרטטו דיאגרמת Gantt וחשבו את זמן הסבב הממוצע (turnaround time).

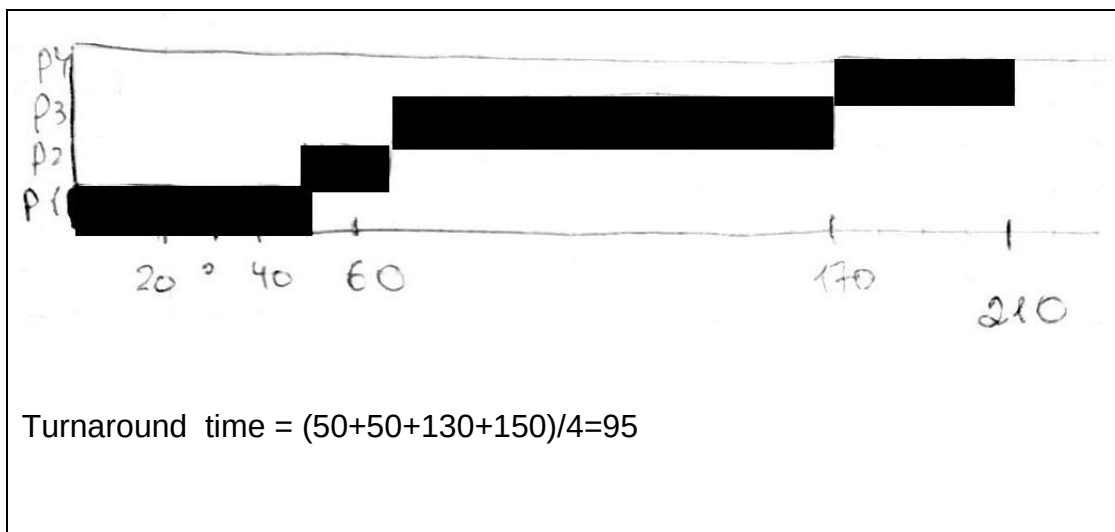
(time quantum = 30 ms) Round Robin .X



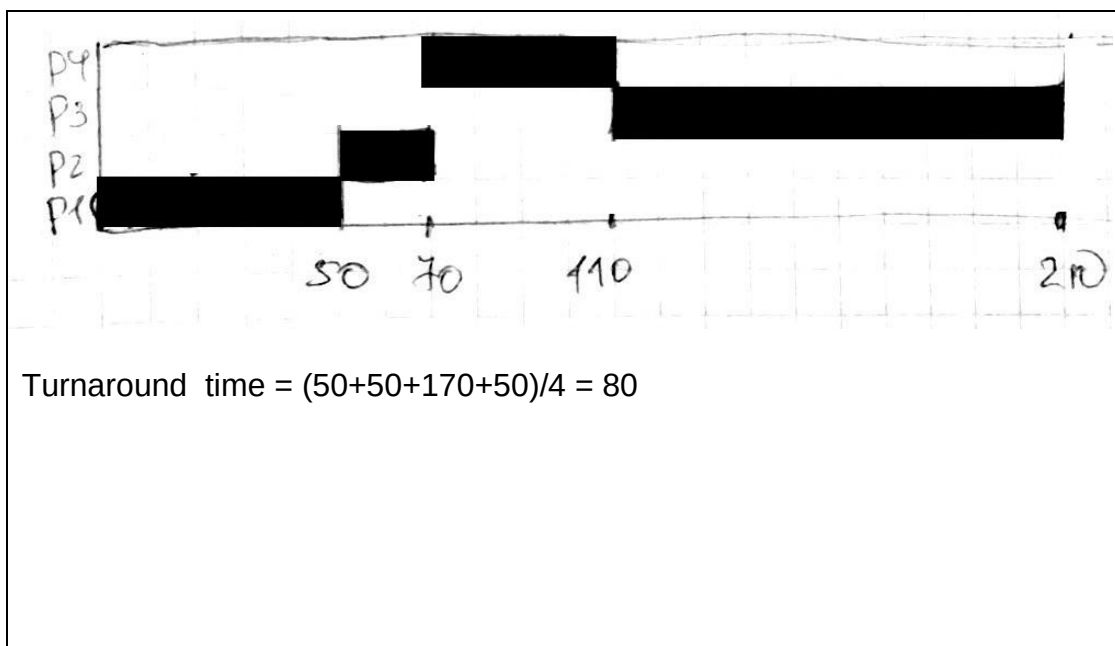
Nonpreemptive Priority Scheduling 1



ג. First-Come, First-Served



ד. Shortest Job First



שאלה 3 (25 נק')

סעיף א' (4 נק')

נתונה מערכת ובה מרחב הזיכרון הווירטואלי הינו של 48 bits, מרחב הזיכרון הפיזי הינו של 32 bits, וגודלו של כל דף הינו 4K. כמה דפים מכילה טבלת הדפים של התהליך? כמה מסגרות (page frames) יש בזיכרון הפיזי? הראו את החישובים.

Virtual memory size is 2^{48}
 Physical memory size is 2^{32}
 Page size = 4K = 2^{12}
 Page table size is $2^{48}/2^{12}=2^{36}$ pages
 Number of frames is $2^{32}/2^{12}=2^{20}$

סעיף ב' (4 נק')

הסבירו מהי טבלת דפים הפוכה (inverted page table)? ציינו יתרון אחד וחסרון אחד של טבלת דפים הפוכה ביחד לטבלת דפים רגילה בת רמה אחת.

טבלת דפים הפוכה (inverted page table) מכילה מיפוי של מסגרות זיכרון לדפים של תהליכים או, במילים אחרות, כל שורה בטבלת דפים הפוכה מתייחסת למסגרת בזיכרון הפיזי ומכילה אינפורמציה לגבי איזה דף ממופה במסגרת ולאיזה תהליך הוא שייך. יתרון:

שימוש בטבלת זיכרון הפוכה חוסך כמות גדולה של זיכרון, שכן זאת טבלה משותפת לכל התהליכים. חסרון:

שורותיה של הטבלה ההפוכה מתייחסות למסגרות זיכרון, כלומר אנחנו לא יכולים להשתמש במספר הדף כאינדקס בטבלת דפים הפוכה. כדי לא לבצע מעבר על כל טבלת הדפים ההפוכה, משתמשים בחיפוש המבוסס על פונקציית ערבול.

סעיף ד' (12 נק')

נתונה מערכת ובה ארבעה page frames. הטבלה הבאה מתארת את מצב הזיכרון הפיזי בזמן נתון.

דף	זמן טעינה	זמן הגישה האחרונה	R bit	Modified bit
1	126	280	1	0
2	230	265	0	1
3	140	270	0	0
4	110	285	1	1

צינו אילו מבין הדפים שלעיל יהיה הראשון שיוחלף על ידי כל אחד מן האלגוריתמים הבאים.

- _____ ³ NRU (a)
 _____ ⁴ FIFO (b)
 _____ ² LRU (c)
 _____ ³ Second-chance FIFO (d)

שאלה 4 (20 נק')

נתונים שני תהליכים A ו-B הרצים במקביל.

בקוד שלפניכם נמחקו הפקודות: `down(sem)`, `up(sem)`, `sem_init(sem, n)`, כאשר `sem` מייצג סמפור ו-n ערך האתחול שלו.

עליכם למקם את הפקודות שנמחקו בצורה כזו שיתקיימו התנאים הבאים:

- (a) קטע הקוד S1 יתבצע לפני קטע הקוד S2
 (b) קטע הקוד S5 יתבצע לפני קטע הקוד S6
 (c) קטע הקוד S6 יתבצע לפני קטע הקוד S7
 (d) קטע הקוד S7 יתבצע לפני קטע הקוד S4

מותר להשתמש בשני סמפורים בלבד. פתרון המשתמש ביותר משני סמפורים יזכה בניקוד חלקי בלבד.

<pre>sem_init(semA, 0) // counting semaphore sem_init(semB, 0) // counting semaphore</pre>	
A	B
<pre>S1 up(SemB) S5 up(SemB) S8 S9 down(SemA) S7 up(SemB)</pre>	<pre>down(SemB) S2 S3 down(SemB) S6 up(SemA) down(SemB) S4</pre>

אין למנוע מקביליות של הרצת הפקודות בתהליכים אלא רק לפי ההתניות a-b-c-d הרשומות!

שאלה 5 (20 נק')

סעיף א' (8 נק')

במערכת יש ארבעה תהליכים וחמישה משאבים. להלן המטריצות Maximum, Allocated ו-Available:

	<i>Allocated</i>	<i>Maximum</i>	<i>Available</i>
Process A	1 0 2 1 1	1 1 2 1 3	0 0 x 1 2
Process B	2 0 1 1 0	2 2 2 1 0	
Process C	1 1 0 1 0	2 1 3 1 0	
Process D	1 1 1 1 0	1 1 2 2 1	

- מטריצת Allocated מתארת הקצאה נוכחית של כל תהליך
- מטריצת Maximum מתארת הקצאה מקסימאלית של כל תהליך
- Available מתארת כמות המשאבים הפנויים מכל סוג.

מהו הערך המינימאלי של x עבורו המערכת נמצאת במצב בטוח (safe state)? הסבירו בקצרה את תשובתכם.

If x is 0, we have a deadlock immediately. If x is 1, process D can run to completion. When it is finished, the available vector is 1 1 2 2 2. Then, we can run A , and available is 2 1 4 3 3. Then, we can run C , and available is 3 2 4 4 3. And, finally, we can run B .

סעיף ב' (9 נק')

בחנו את הקוד הבא וענו על השאלות שבהמשך:

flag[2] = {false, false};	
Process 0:	Process 1:
<pre>while(true){ flag[0] = true; while(flag[1]){ flag[0] = false; delay a short time flag[0] = true; } <critical section> flag[0] = false; }</pre>	<pre>while(true){ flag[1] = true; while(flag[0]){ flag[1] = false; delay a short time flag[1] = true; } <critical section> flag[1] = false; }</pre>

	}
--	---

לכל סעיף הקיפו את התשובה הנכונה.

- א. הקוד מקיים מניעה הדדית (mutual exclusion). **כן** / לא
- ב. עלול להיות קיפאון (deadlock). **כן** / לא
- ג. עלולה להיות הרעבה. **כן** / לא

סעיף ג' (3 נק')

האם תהליך שמבצע לולאה אינסופית נמצא במצב הרעבה? הסבירו בקצרה את תשובתכם.

לא, תהליך כזה לא נמצא במצב הרעבה.
הרעבה היא מצב שבו תהליך שלא נמצא בקיפאון אינו מקבל שירות (משאב)
שהוא ביקש במשך פרק זמן ארוך מדי ביחס לזמן התגובה הנדרש ממנו
(תיאורטית – גם זמן אינסופי), משום שהשירות ניתן תמיד לתהליכים אחרים לפניו.
תהליך שמבצע לולאה אינסופית כן מקבל CPU.